8、多按键输入驱动

全志V3S嵌入式驱动开发（多按键输入驱动）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2024-01-20 23:08:13 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量982 Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S开发板上的按键驱动开发过程，从查找电路图、设备树到驱动代码分析，以及如何解决在测试中遇到的按键值错误问题。通过对设备树、驱动代码的修改，成功实现了按键输入的正确识别。

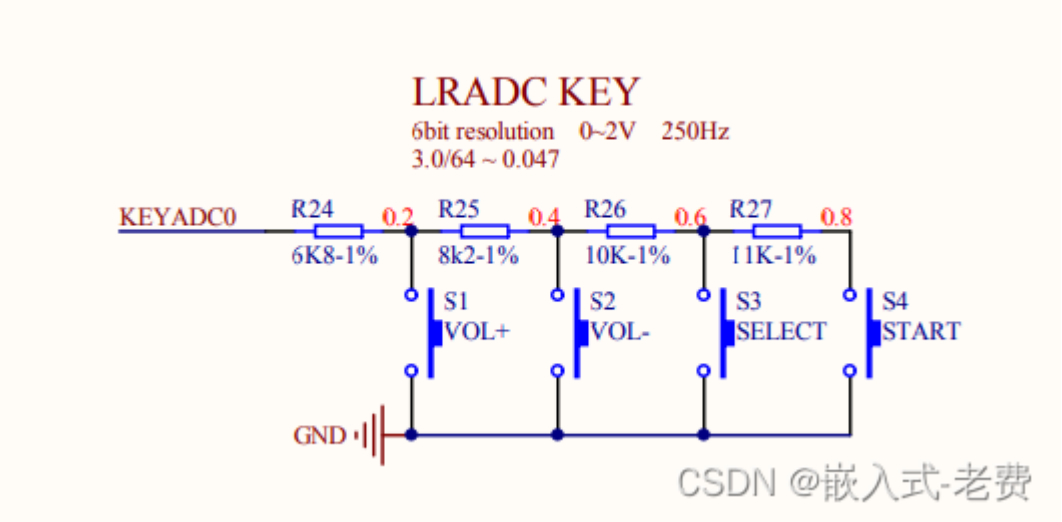
摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

前面我们说过，荔枝派的开发板上面，有4个按键本身不是通过gpio连接到soc上面的。它是通过ad的方法，连接到主芯片的。这个时候，不同的按键被按下的时候，就会生成不同的电压或者电流，那么完全可以根据对应的电信号，推算出当前是哪一个按键被按下去了。

1、查找电路图

Open: 5f2810a1359107d59bdf709dc043ffbb_MD5.png



简单看一下电路之后，下面就是去找设备树，对应的信号是什么、在哪里。

2、查找设备树

在sun8i-v3s-licheepi-zero-dock.dts文件当中，我们发现了这样的内容，

```
&lradc {
    vref-supply = <&reg_vcc3v0>;
    status = "okay";

    button@200 {
        label = "Volume Up";
        linux,code = <KEY_VOLUMEUP>;
        channel = <0>;
        voltage = <200000>;
    };

    button@400 {
        label = "Volume Down";
        linux,code = <KEY_VOLUMEDOWN>;
        channel = <0>;
        voltage = <400000>;
    };

    button@600 {
        label = "Select";
        linux,code = <KEY_SELECT>;
        channel = <0>;
        voltage = <600000>;
    };

    button@800 {
        label = "Start";
        linux,code = <KEY_OK>;
        channel = <0>;
        voltage = <800000>;
    };
};
```

很明显，每一个button都是和电路中的按键是一一对应的，这个没有问题。那么，我们不禁还有一个疑问，既然是ad转换得到的结果，那么肯定要知道ad相关的设备配置是恶还那么。仔细找了一下，可以在sun8i-v3s.dtsi文件发现这样的内容，

```
        lradc: lradc@01c22800 {
            compatible = "allwinner,sun4i-a10-lradc-keys";
            reg = <0x01c22800 0x400>;
            interrupts = <GIC_SPI 30 IRQ_TYPE_LEVEL_HIGH>;
            status = "disabled";
        };
```

看到这里，大家应该放心了，确实是有这么一个ad的驱动。兼容的设备是sun4i-a10-lradc-keys，寄存器地址空间是0x01c22800，长度是0x400，中断是GIC_SPI类型，状态关闭。有了设备树，还有了兼容设备号，接下来的一步就是根据设备号sun4i-a10-lradc-keys找到对应的驱动文件。

3、查找驱动代码，准备测试程序

通过工具查找一下，不难发现，文件在这，即sun4i-lradc-keys.c，

```
static const struct of_device_id sun4i_lradc_of_match[] = {
    { .compatible = "allwinner,sun4i-a10-lradc-keys", },
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, sun4i_lradc_of_match);

static struct platform_driver sun4i_lradc_driver = {
    .driver = {
        .name   = "sun4i-a10-lradc-keys",
        .of_match_table = of_match_ptr(sun4i_lradc_of_match),
    },
    .probe  = sun4i_lradc_probe,
};

module_platform_driver(sun4i_lradc_driver);

MODULE_DESCRIPTION("Allwinner sun4i low res adc attached tablet keys
driver");
MODULE_AUTHOR("Hans de Goede <hdegoede@redhat.com>");
MODULE_LICENSE("GPL");
```

大致看一下最后一段的代码，内容方面应该错不了。一般来说，如果按键ok的话，会在设备启动的时候生成一个/dev/input/event0节点，此时，如果编写一个应用程序，

读写这些节点，就完全可以获取相关的按键信息。所以，我们还得准备一个input.c的读写程序，

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>
#include <linux/input.h>
#include <linux/input-event-codes.h>

const char * path = "/dev/input/event0";

int main(char argc,char *argv[])
{
   int ret;

   int fd;
   struct input_event event;

   fd = open(path,O_RDONLY);
   if(fd < 0)
   {
      perror(path);
      exit(-1);
   }

   while(1)
   {
      ret = read(fd,&event,sizeof(struct input_event));
      if(ret == sizeof(struct input_event))
      {
            if(event.type != EV_SYN)
            {
               printf("Event: time
%ld.%ld,",event.time.tv_sec,event.time.tv_usec);
               printf("type:%d,code:%d,value:%d\n",
event.type,event.code,event.value);
```

```
            }
        }
    }

    close(fd);

    return 0;
}
```

上面这段代码是从其他地方copy而来，谢谢了。准备好了程序之后，下面就是交叉编译，下载到开发板上面。但是实际运行的时候，发现按键被按下的时候，有三个按键的数值居然是一样的，都是352，另外一个是114。这就非常蹊跷了。

4、发现问题、解决问题

一般情况下，遇到这种情况，第一个怀疑的就是电阻坏了，R24、R25、R26、R27当中肯定有三个被击穿了，不然这种情况是说不过去的。于是，我们拿掉sd卡，让v3s继续跑之前norflash里面的嵌入式系统，输入key程序，也就是按键测试程序，没想到结果居然是正常的。这说明，硬件，没问题。问题出在上层应用或者驱动程序。

回过头查看sun4i-lradc-keys.c，惊讶地发现电压判断标准是根据sun8i-v3s-licheepi-zero-dock.dts中的voltage来验证的，这并不符合实际的情况。我们通过printk&dmesg打印，也验证了这一想法，所以如果需要得到正确的按键数值，只需要修正一下sun4i-lradc-keys.c中的判断逻辑就可以了，修改方法如下，具体的标定数值可以做实验来解决，

```
#if 0 // by feixiaoxing
        voltage = val * lradc->vref / 63;

        for (i = 0; i < lradc->chan0_map_count; i++) {
            diff = abs(lradc->chan0_map[i].voltage - voltage);
            if (diff < closest) {
                closest = diff;
                keycode = lradc->chan0_map[i].keycode;
            }
        }
#else
        printk("val = %d\n", val);
        if(val >=9 && val <= 13)
            keycode = lradc->chan0_map[0].keycode;
        else if(val >=24 && val <= 29)
```

```
            keycode = lradc->chan0_map[1].keycode;
        else if(val >= 35 && val <= 40)
            keycode = lradc->chan0_map[2].keycode;
        else
            keycode = lradc->chan0_map[3].keycode;
#endif
```

经过这一次修改，我们重新编译kernel内核，烧入zImage，启动后输入key程序，这样就得到了我们想要的最终结果，即稳定地输出按键值，

```
# ./input
[    8.623196] val = 10
Event: time 47.807318,type:1,code:115,value:1
Event: time 48.78567,type:1,code:115,value:0
[   10.920313] val = 26
Event: time 50.104437,type:1,code:114,value:1
Event: time 50.340531,type:1,code:114,value:0
[   12.709584] val = 37
Event: time 51.893708,type:1,code:353,value:1
Event: time 52.141508,type:1,code:353,value:0
[   14.549618] val = 45
Event: time 53.733726,type:1,code:352,value:1
Event: time 53.969827,type:1,code:352,value:0
[   16.186513] val = 11
Event: time 55.370623,type:1,code:115,value:1
Event: time 55.645799,type:1,code:115,value:0
[   17.514789] val = 27
Event: time 56.698907,type:1,code:114,value:1
Event: time 56.950631,type:1,code:114,value:0
[   20.823740] val = 39
Event: time 60.7851,type:1,code:353,value:1
Event: time 60.279112,type:1,code:353,value:0
[   21.987936] val = 44
Event: time 61.172044,type:1,code:352,value:1
Event: time 61.435486,type:1,code:352,value:0
```